Lecture 14 – Binary Search in Array

DSA MASTERY SERIES | LeetCode 704, 34, 35 | C++ Focused | Print-Ready A4

0. Why Binary Search?

THE PROBLEM THAT CREATED BINARY SEARCH

Socho: Ek dictionary me ek word dhoondhna hai. Agar tum pehle page se shuru karke ek-ek page turn karoge (Linear Search), to 1000 pages ki dictionary me average 500 pages dekhne padenge. Ye bahut slow hai!

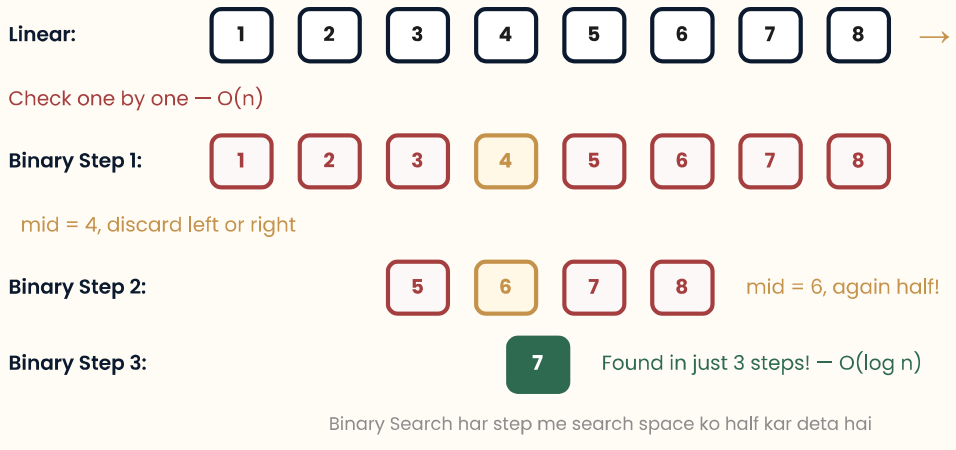
Lekin agar tum **middle page** kholo aur dekho ki word alphabetically usse pehle hai ya baad me, to tum **aadhe pages ko turant discard** kar sakte ho. Baar-baar aisa karte raho — har baar search space half ho jata hai. Bas 10 steps me 1000 pages se 1 page par aa jate ho!

Yehi Binary Search ka core idea hai — "Divide and Conquer".

REAL-WORLD USE CASES

- **Database Indexing:** MySQL, PostgreSQL me B-Trees use hote hain — Binary Search ka advanced version. Billions of records me microseconds me data mil jata hai.
- **Git Bisect:** Git me `git bisect` command se bug wala commit dhundhte hain — har baar commit history half karke search karta hai.
- **Auto-Complete / Spell Check:** Sorted dictionary me word search — Binary Search se O(log n) me result.
- **IP Routing:** Routers sorted routing tables me destination IP search karte hain using Binary Search.
- **Compilers:** Symbol tables (variable names) ko sorted rakha jata hai — Binary Search se fast lookup.

Visual: Linear Search vs Binary Search — Search Space Reduction



CRITICAL: WHY (LOW + HIGH) / 2 IS DANGEROUS

Jab `low` aur `high` bahut bade integers hote hain (jaise `INT_MAX` ke paas), to `low + high` ka result **integer overflow** kar sakta hai.

Example: `low = 2,000,000,000` aur `high = 2,000,000,000`

`low + high = 4,000,000,000 → INT_MAX (2,147,483,647) se bada! → Overflow → Negative value!`

Safe Formula: `mid = low + (high - low) / 2`

Isme `high - low` kabhi overflow nahi karega kyunki `high >= low` hamesha hota hai. Ye formula mathematically same hai:

`low + (high - low) / 2 = (2*low + high - low) / 2 = (low + high) / 2`

WHILE(START <= END) VS WHILE(START < END)

Condition	Use When	Behavior
`while (start <= end)`	Target **exactly find** karna hai (LeetCode 704)	Loop tab tak chalega jab `start == end` (single element bachi ho). Agar target nahi mila, loop end hone ke baad `start > end` hoga.
`while (start < end)`	**Lower Bound / Upper Bound** nikaalna hai (LeetCode 35)	Loop `start == end` hone se pehle hi ruk jata hai. Final answer `start` (ya `end`) me hota hai. Boundary search ke liye best hai.

Rule of Thumb: Agar exact index return karna hai (found / not found), to <= use karo. Agar insertion position ya boundary nikaalni hai, to < use karo.

◈ ◈ ◈

1. LeetCode 704 – Binary Search

PROBLEM STATEMENT

Ek **sorted (ascending)** integer array `nums` aur integer `target` diya gaya hai. Agar `target` array me exist karta hai, to uska **index return karo**. Nahi to **-1 return karo**.

Example 1: nums = [-1,0,3,5,9,12], target = 9 → Output: **4**

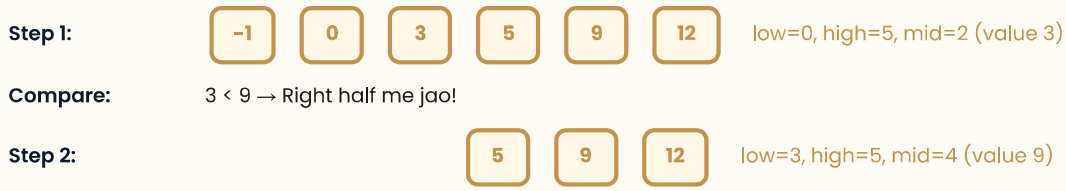
Example 2: nums = [-1,0,3,5,9,12], target = 2 → Output: **-1**

Example 3: nums = [5], target = 5 → Output: **0**

ALGORITHM WORKFLOW

1. Do pointers initialize karo: `low = 0, high = n - 1`
2. Middle index nikalo: `mid = low + (high - low) / 2` (overflow safe)
3. `nums[mid]` ko `target` se compare karo:
   - `nums[mid] == target` → **Return mid** (found!)
   - `nums[mid] < target` → **low = mid + 1** (right half me search karo)
   - `nums[mid] > target` → **high = mid - 1** (left half me search karo)
4. Step 2-3 tab tak repeat karo jab tak `low <= high`
5. Loop end hone par target nahi mila → **Return -1**

Visual: Binary Search on nums = [-1, 0, 3, 5, 9, 12], target = 9



Sirf 2 comparisons me target mil gaya — O(log n) power!

C++ Implementation

```cpp
// LeetCode 704 - Binary Search
class Solution {
public:
    int search(vector<int>& nums, int target) {

        int low = 0;
        int high = nums.size() - 1;

        while (low <= high) {

            // Overflow-safe middle index
            int mid = low + (high - low) / 2;

            // Target Found
            if (nums[mid] == target)
                return mid;

            // Target right half me hoga
            if (nums[mid] < target)
                low = mid + 1;

            // Target left half me hoga
            else
                high = mid - 1;
        }

        // Target Not Found
        return -1;
    }
};
```

STEP-BY-STEP DRY RUN

Input: nums = [-1, 0, 3, 5, 9, 12], target = 9

Iteration 1: low=0, high=5 → mid = 0 + (5-0)/2 = **2** → nums[2] = 3 → 3 < 9 → **low = 3**

Iteration 2: low=3, high=5 → mid = 3 + (5-3)/2 = **4** → nums[4] = 9 → 9 == 9 → **Return 4**

Total comparisons: 2 → O(log 6) ≈ 2.58 ✓

COMPLEXITY ANALYSIS

Case	Time	Reason
Best Case	O(1)	Target exactly middle me ho — ek hi comparison
Average Case	O(log n)	Har step me search space half hota hai — $\log_2(n)$ steps
Worst Case	O(log n)	Target last possible position par ya absent ho

Space Complexity: O(1) — Sirf 3 variables (low, high, mid) use hote hain.

◈ ◈ ◈

2. LeetCode 34 – Find First and Last Position

PROBLEM STATEMENT

Ek **sorted** integer array `nums` aur `target` diya gaya hai. Target ka **first aur last occurrence** return karo. Agar target nahi mila, to `[-1, -1]` return karo.

Example 1: nums = [5,7,7,8,8,10], target = 8 → Output: **[3, 4]**

Example 2: nums = [5,7,7,8,8,10], target = 6 → Output: **[-1, -1]**

Example 3: nums = [2,2], target = 2 → Output: **[0, 1]**

INTUITION & REAL-LIFE ANALOGY

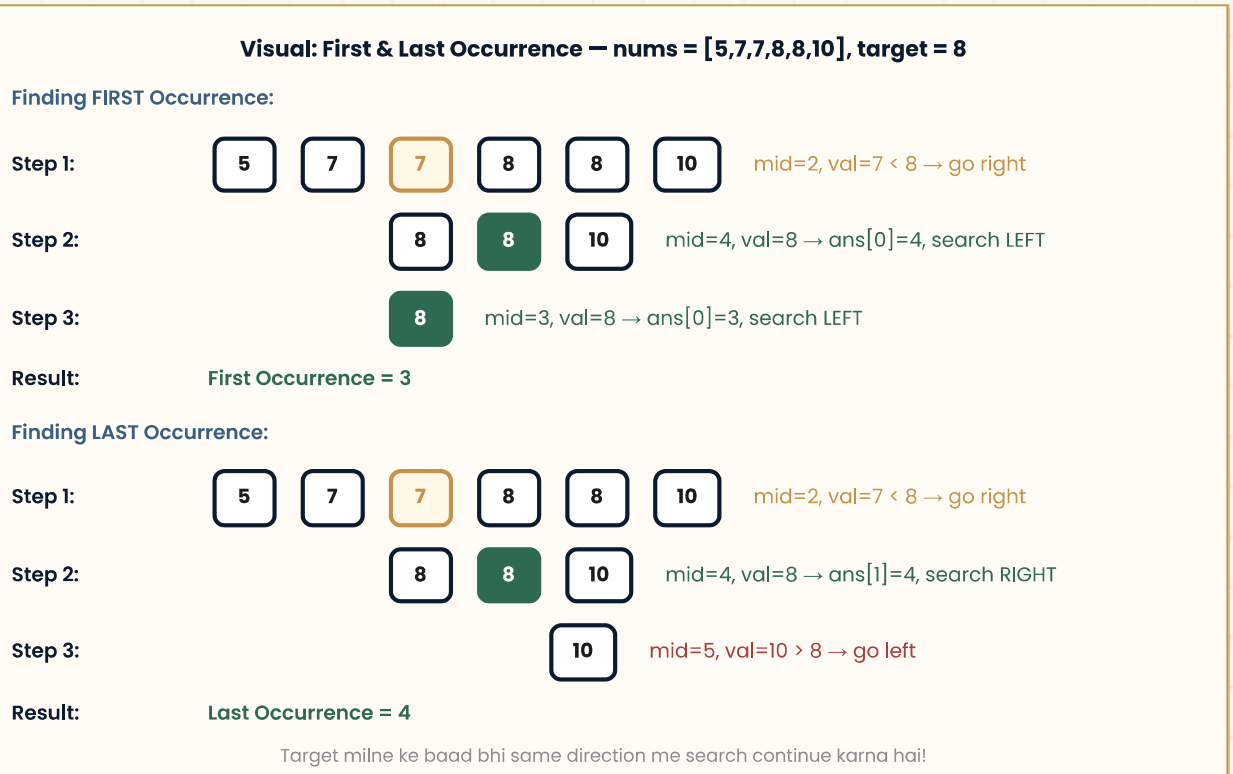
Analogy: Socho ek phone directory me "Sharma" naam ke kitne log hain aur sabse pehla aur sabse aakhri "Sharma" kahan hai. Tum pehle first "Sharma" dhundhoge — jab mil jaye, to uske **left side** aur check karoge ki koi aur "Sharma" hai kya. Phir last "Sharma" dhundhoge — jab mil jaye, to uske **right side** check karoge.

Binary Search **do baar** chalana hai — ek first occurrence ke liye, ek last occurrence ke liye. Target milne par turant return nahi karna; instead, **answer store karo aur usi direction me search continue karo**.

ALGORITHM WORKFLOW

1. Answer array initialize karo: `ans = [-1, -1]`
2. **First Occurrence ke liye Binary Search:**
   - `nums[mid] == target` → `ans[0] = mid` store karo, **left me aur search karo** (`end = mid - 1`)
   - `nums[mid] < target` → `start = mid + 1`
   - `nums[mid] > target` → `end = mid - 1`
3. Search space reset karo: `start = 0, end = n - 1`
4. **Last Occurrence ke liye Binary Search:**
   - `nums[mid] == target` → `ans[1] = mid` store karo, **right me aur search karo** (`start = mid + 1`)
   - `nums[mid] < target` → `start = mid + 1`
   - `nums[mid] > target` → `end = mid - 1`
5. ans return karo

Visual: First & Last Occurrence — nums = [5,7,7,8,8,10], target = 8



Target milne ke baad bhi same direction me search continue karna hai!

C++ Implementation

```cpp
// LeetCode 34 - Find First and Last Position
class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {

        vector<int> ans(2, -1);

        int start = 0;
        int end = nums.size() - 1;

        // ========== First Occurrence ==========
        while (start <= end) {
            int mid = start + (end - start) / 2;

            if (nums[mid] == target) {
                ans[0] = mid;       // Store answer
                end = mid - 1;      // But keep searching LEFT
            }
            else if (nums[mid] < target) {
                start = mid + 1;
            }
            else {
                end = mid - 1;
            }
        }

        // Reset search space for last occurrence
        start = 0;
        end = nums.size() - 1;

        // ========== Last Occurrence ==========
        while (start <= end) {
            int mid = start + (end - start) / 2;

            if (nums[mid] == target) {
                ans[1] = mid;       // Store answer
                start = mid + 1;    // But keep searching RIGHT
            }
            else if (nums[mid] < target) {
                start = mid + 1;
            }
            else {
                end = mid - 1;
            }
        }

        return ans;
    }
};
```

STEP-BY-STEP DRY RUN

Input: nums = [5,7,7,8,8,10], target = 8

First Occurrence:

start=0, end=5 → mid=2, nums[2]=7 < 8 → start=3

start=3, end=5 → mid=4, nums[4]=8 == 8 → **ans[0]=4**, end=3

start=3, end=3 → mid=3, nums[3]=8 == 8 → **ans[0]=3**, end=2

start=3, end=2 → Loop ends → **First = 3**

Last Occurrence:

start=0, end=5 → mid=2, nums[2]=7 < 8 → start=3

start=3, end=5 → mid=4, nums[4]=8 == 8 → **ans[1]=4**, start=5

start=5, end=5 → mid=5, nums[5]=10 > 8 → end=4

start=5, end=4 → Loop ends → **Last = 4**

Final Answer: [3, 4]

COMPLEXITY ANALYSIS

Time Complexity: O(log n) + O(log n) = **O(log n)** — Do binary searches chal rahe hain, dono independent.

Space Complexity: O(1) — Sirf answer array aur pointers use hote hain.

EDGE CASES & GOTCHAS

- **Empty array:** `nums.size() == 0` → dono loops nahi chalenge, `[-1, -1]` return hoga.
- **Single element:** `nums = [2], target = 2` → First = Last = 0. Reset ke baad second loop bhi same result dega.
- **All elements same:** `nums = [2,2,2,2]` → First loop left tak jayega (ans[0]=0), last loop right tak jayega (ans[1]=3).
- **Target not present:** Dono loops me `ans` update nahi hoga → `[-1, -1]` return hoga.

◈ ◈ ◈

3. LeetCode 35 – Search Insert Position

PROBLEM STATEMENT

Ek **sorted array of distinct integers** `nums` aur `target` diya gaya hai. Agar target mil jaye, to uska **index return karo**. Nahi to wo **index return karo** jahan target insert hone par array sorted rahega.

Example 1: nums = [1,3,5,6], target = 5 → Output: **2**

Example 2: nums = [1,3,5,6], target = 2 → Output: **1** (2 ko 1 aur 3 ke beech insert karna hai)

Example 3: nums = [1,3,5,6], target = 7 → Output: **4** (sabse end me insert karna hai)

INTUITION & REAL-LIFE ANALOGY

Analogy: Socho tumhe ek already sorted bookshelf me ek nayi book rakhni hai. Tum middle se shuru karke dekhte ho ki book alphabetically usse pehle hai ya baad me. Har baar aadhi shelf discard kar dete ho. Aakhir me tumhe exact position mil jati hai jahan book rakhni hai — chahe wahan pehle se book ho ya na ho.

Ye problem basically **Lower Bound** nikaalna hai — **pehla element jo target se bada ya equal hai**. Binary Search se ye O(log n) me mil jata hai.

ALGORITHM WORKFLOW

1. Default answer set karo: `answer = nums.size()` (agar target sabse bada hua, to end me insert hoga)
2. Binary Search start karo: `start = 0, end = n - 1`
3. `nums[mid] >= target` → Ye possible answer hai!
   - `answer = mid` store karo
   - **Left me aur search karo** — shayad aur chhota valid index ho (`end = mid - 1`)
4. `nums[mid] < target` → Right me search karo (`start = mid + 1`)
5. Loop end hone par `answer` return karo

Visual: Search Insert Position — nums = [1,3,5,6], target = 2

Initial:	1 3 5 6	ans = 4 (default: end me insert)
Step 1:	1 **3** 5 6	mid=1, val=3 ≥ 2 → ans=1, go LEFT
Step 2:	**1**	mid=0, val=1 < 2 → go RIGHT
Step 3:		start=1, end=0 → Loop ends
Answer:		**Return ans = 1 → 2 should be inserted at index 1**

Lower Bound: First element ≥ target

C++ Implementation

```cpp
// LeetCode 35 - Search Insert Position
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {

        int start = 0;
        int end = nums.size() - 1;

        // Default: target sabse bada hai → end me insert
        int answer = nums.size();

        while (start <= end) {

            int mid = start + (end - start) / 2;

            // Possible answer found!
            if (nums[mid] >= target) {
                answer = mid;           // Store as potential answer
                end = mid - 1;      // But check left for smaller index
            }
            else {
                // Target abhi bhi bada hai → right me jao
                start = mid + 1;
            }
        }

        return answer;
    }
};
```

STEP-BY-STEP DRY RUN

Input: nums = [1,3,5,6], target = 2

Initial: start=0, end=3, answer=4 (default)

Iter 1: mid=1, nums[1]=3 ≥ 2 → **answer=1**, end=0 (left me aur check karo)

Iter 2: start=0, end=0 → mid=0, nums[0]=1 < 2 → start=1 (right me jao)

End: start=1, end=0 → Loop ends → **Return answer = 1**

Index 1 par 3 hai, 2 ko 1 ke baad insert karna hai → [1, **2**, 3, 5, 6]

COMPLEXITY ANALYSIS

Case	Time	Reason
Best Case	O(1)	Target exactly middle me ho
Average / Worst	O(log n)	Binary Search — har step me half

Space Complexity: O(1) — Sirf 3 variables.

EDGE CASES & GOTCHAS

- **Target sabse chhota ho:** `nums = [3,5,6], target = 1` → answer = 0 (start me insert)
- **Target sabse bada ho:** `nums = [1,3,5], target = 7` → answer = 3 (default, end me insert)
- **Target already present ho:** `nums = [1,3,5,6], target = 5` → answer = 2 (same index return)
- **Empty array:** `nums = [], target = 1` → answer = 0 (default size)

4. Pattern Summary & Comparison

BINARY SEARCH VARIATIONS AT A GLANCE

Problem	Condition on Match	Return What	Loop Condition
Standard Search (704)	`nums[mid] == target` → **return mid**	Index or -1	`start <= end`
First Occurrence (34)	`nums[mid] == target` → **store & go LEFT**	Leftmost index	`start <= end`
Last Occurrence (34)	`nums[mid] == target` → **store & go RIGHT**	Rightmost index	`start <= end`
Lower Bound (35)	`nums[mid] >= target` → **store & go LEFT**	Insert position	`start <= end`

PATTERN TRANSFER & RELATED PROBLEMS

- **LeetCode 704** — Binary Search (Base)
- **LeetCode 34** — First & Last Position (Boundaries)
- **LeetCode 35** — Search Insert Position (Lower Bound)
- **LeetCode 69** — Sqrt(x) (Binary Search on answer)
- **LeetCode 153** — Find Minimum in Rotated Sorted Array (Modified BS)
- **LeetCode 33** — Search in Rotated Sorted Array (Modified BS)
- **LeetCode 4** — Median of Two Sorted Arrays (Binary Search on partitions)
- **LeetCode 875** — Koko Eating Bananas (Binary Search on answer space)

KEY TAKEAWAYS

- **Binary Search sirf sorted data pe kaam karta hai.** Agar array sorted nahi hai, pehle sort karo (O(n log n)) ya different approach socho.
- **Mid formula:** Hamesha `low + (high - low) / 2` use karo — integer overflow se bachne ke liye.
- **While condition:** Exact index chahiye → <=; Boundary/position chahiye → < bhi chal sakta hai depending on implementation.
- **Target milne ke baad bhi search continue** karna padta hai jab first/last occurrence ya lower bound nikaalna ho.
- **O(log n)** ka matlab: 1 billion elements me sirf ~30 comparisons!

QUICK REVISION CHEATSHEET

Why Binary Search?
Sorted data me O(log n) search.
1 Billion elements = 30 steps only!
Real use: DB indexing, Git bisect,
IP routing, auto-complete

Mid Formula (CRITICAL)
Wrong: `(low + high) / 2`
→ Integer overflow risk!
Correct: `low + (high-low)/2`
→ Always safe, same result

While(start <= end) vs <
<=: Exact index return karna ho
→ LeetCode 704, 34

<: Boundary/position nikaalna ho
→ Lower bound, upper bound patterns

Rule: Exact → <= | Boundary → <

Three Problems Pattern
704: Match → return immediately
34-First: Match → store, go LEFT
34-Last: Match → store, go RIGHT
35: >= target → store, go LEFT
(Lower Bound pattern)

Common Mistakes
× (low+high)/2 → overflow
× First/Last me match pe return kar dena
× Loop condition galat → infinite loop
× mid-1 ya mid+1 bhool jana
× Array unsorted hone par BS lagana

Interview Questions
Q1: BS kyu O(log n)?
→ Har step search space half

Q2: Overflow kaise avoid karein?
→ low + (high-low)/2

Q3: First/Last occurrence ka logic?
→ Match pe store karo, same dir me continue

Q4: Rotated array me BS?
→ Check which half is sorted

Harshal Chauhan